

Compiladores - Teste 5 - AST

Total de pontos 6/6

O e-mail do participante (**gustavo.gripaldi@iesb.edu.br**) foi registrado durante o envio deste formulário.

Qual é o papel principal do Analisador Sintático (Parser) em relação à AST? * 1/1

- ☐ Analisar os tipos de dados das variáveis dentro da AST e emitir mensagens de erro de tipagem antes de verificar a sintaxe.
- ☐ Percorrer o texto caractere por caractere para identificar palavras reservadas usando expressões regulares antes de criar a AST.
- ☒ Ler a sequência de tokens fornecida pelo analisador léxico, validar se ela respeita as regras da gramática (BNF) e construir a estrutura da AST na memória.
- ☐ Traduzir os nós da AST diretamente para código de máquina executável no barramento do processador sem etapas intermediárias.
- ☐ Converter o arquivo de código-fonte diretamente em uma imagem vetorial para visualização gráfica das funções.

Ao analisar uma expressão matemática como $2 + 3 * 4$, como a precedência dos operadores se reflete na estrutura da AST? *1/1

- ☒ O operador de maior precedência (*) fica em um nível mais profundo da árvore, sendo filho da operação de menor precedência (+), garantindo que seja avaliado primeiro.
- ☐ O operador de menor precedência (+) fica posicionado em um nível mais profundo para ser processado antes dos operandos vizinhos.
- ☐ A precedência não altera a forma da AST; a árvore é gerada idêntica para qualquer combinação de operadores, deixando o cálculo para o sistema operacional.
- ☐ Todos os operadores e operandos são alinhados como nós irmãos no mesmo nível da árvore, e a ordem de execução é definida estritamente da esquerda para a direita.
- ☐ Os números (2, 3, 4) tornam-se nós raiz da árvore, enquanto os operadores (+, *) ficam armazenados como folhas na extremidade inferior.



Por que elementos sintáticos como chaves {} e parênteses () geralmente não *1/1 aparecem como nós explícitos em uma AST?

- ☐ Porque parênteses e chaves são utilizados apenas em linguagens interpretadas, não sendo necessários em compiladores baseados em AST.
- ☒ Porque o agrupamento, o escopo e a hierarquia dos blocos de código já estão implicitamente definidos pela própria estrutura de nós pai e filhos da árvore.
- ☐ Porque o analisador léxico ignora completamente qualquer caractere que não seja uma letra ou um número.
- ☐ Porque eles são convertidos automaticamente em números inteiros na tabela de símbolos antes da fase de parsing.
- ☐ Porque esses símbolos são apagados pelo analisador sintático para economizar memória RAM durante o processo de compilação.

Após a geração bem-sucedida da AST pelo Parser, qual é a etapa seguinte em *1/1 que essa estrutura de árvore é utilizada pelo compilador/transpilador?

- ☐ A árvore é enviada de volta ao Lexer para passar por uma segunda fase de tokenização e simplificação de texto.
- ☒ A árvore é percorrida por um avaliador, analisador semântico ou gerador de código para executar as instruções ou traduzi-las para a linguagem de destino.
- ☐ A árvore é descartada e o compilador volta a ler o texto do código-fonte original para gerar os arquivos binários.
- ☐ A árvore é convertida em um arquivo executável .exe de forma direta, sem a necessidade de checar tipos ou escopos.
- ☐ A árvore é salva em um banco de dados relacional para que o interpretador possa consultar as instruções via SQL.



Considere a classe Python NoAST abaixo, usada para estruturar os nós de uma árvore sintática: *1/1

class NoAST:

def __init__(self, tipo, valor=None, filhos=None):

self.tipo = tipo

self.valor = valor

self.filhos = filhos if filhos is not None else []

Na construção dessa estrutura, qual é a função das propriedades tipo, valor e filhos?

- ☐ tipo define o tipo de dado em C (ex: "int"), valor armazena o resultado do cálculo e filhos guarda a linha e coluna do arquivo.
- ☐ tipo indica se o nó é raiz ou folha, valor determina a quantidade de memória em bytes e filhos armazena o endereço de ponteiro do nó pai.
- ☒ tipo identifica o papel sintático do nó (ex: "CONDICIONAL"), valor guarda dados literais ou operadores (ex: "+", "10") e filhos armazena a lista de sub-nós dependentes.
- ☐ tipo armazena a expressão regular do token, valor indica a ordem na pilha e filhos contém o código traduzido para C.
- ☐ tipo serve para nomear a variável, valor armazena a palavra-chave em maiúsculas e filhos é um dicionário com os erros de sintaxe.



O que é uma Árvore de Sintaxe Abstrata (AST) no contexto da construção de compiladores? *1/1

- ☒ Uma representação em árvore hierárquica e simplificada do código-fonte, focada na estrutura lógica do programa e desprovida de detalhes sintáticos irrelevantes como parênteses, chaves e vírgulas.
- ☐ Um gráfico de dependências de memória utilizado no momento da execução para alocar variáveis globais e locais na pilha (stack).
- ☐ Uma tabela Hash que armazena os endereços de memória, escopos e tipos de todos os identificadores declarados no arquivo.
- ☐ Uma lista encadeada de tokens gerada diretamente pelo analisador léxico para representar a sequência linear de palavras reservadas do programa.
- ☐ Uma representação gráfica exata e completa do código contendo cada caractere lido do arquivo, incluindo espaços em branco, quebras de linha e comentários.

Este formulário foi criado fora de seu domínio. - [Entre em contato com o proprietário do formulário](#) - [Termos de Serviço](#) - [Política de Privacidade](#)

Este formulário parece suspeito? [Denunciar](#)

Google Formulários



